01 Spotify ETL Project

This project is designed to bring together everything we've learned in the course, from understanding variables and data structures like lists and dictionaries, to mastering loops, object-oriented programming, and working with date/time and various file formats. Our goal is to apply these skills to build a practical data pipeline.

Project Title: Building a Serverless Spotify ETL Data Pipeline using Python and AWS

As data engineers, building ETL (Extract, Transform, Load) pipelines is a core responsibility. This project will provide hands-on experience with this fundamental task.

1. Understanding the Business Problem

The primary goal of any data project is to **generate value for the business by finding insights from collected data**. For this project, our client is deeply passionate about the music industry and seeks to understand it better by analyzing data to discover patterns and inform music decisions.

Our client wants to start by focusing on the **top global songs playlist available on Spotify Billboard**, which updates weekly. The objective is to **collect this data on a weekly basis to build a large dataset over one to two years**. This extensive dataset will then enable the client to identify trends such as:

- What types of songs are trending.
- What kind of albums are trending.
- Who are the artists behind these popular songs.

To achieve this, we have been tasked with building an **ETL pipeline**.

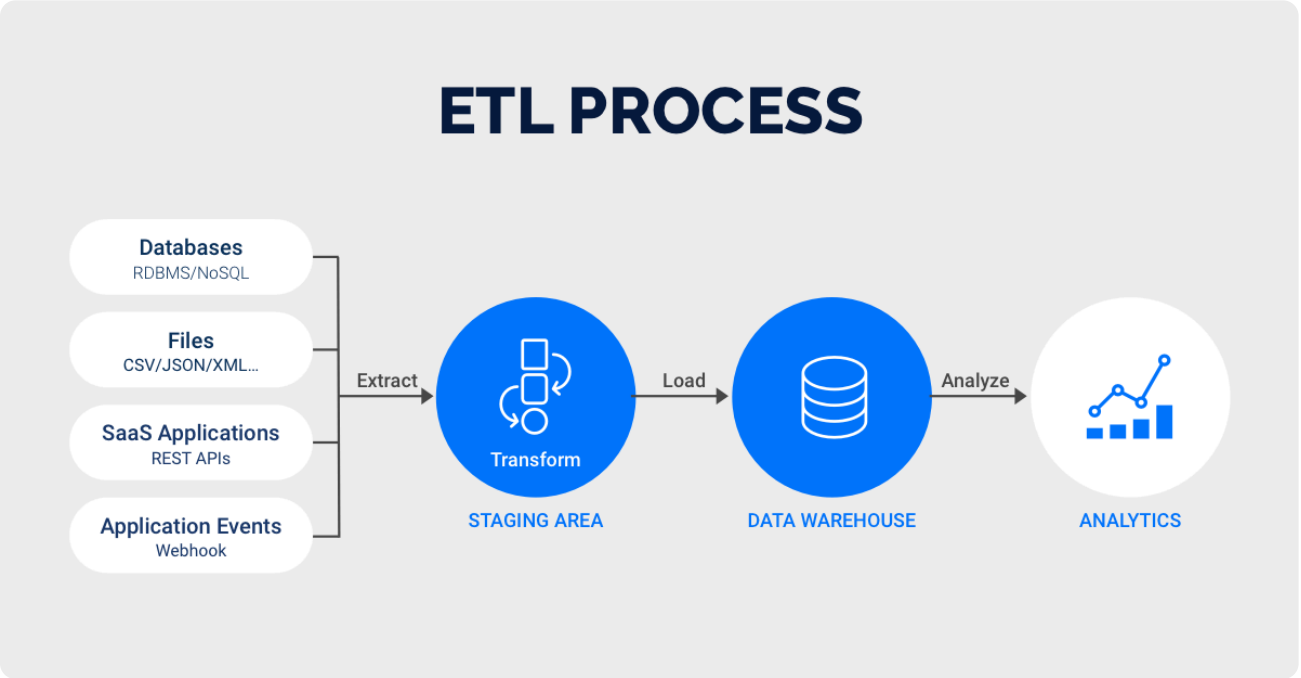
2. What is ETL?

ETL stands for **Extract, Transform, Load**. Let's break down each component:

- **Extract**: This phase involves **collecting raw data from multiple sources**. Data can originate from various places, including APIs (like the Spotify API in our case), Relational Database Management Systems (RDBMS), analytics data, sensor data, and more.
- **Transform**: Once extracted, data often needs to be refined. The transformation phase involves **applying logic to clean the data, add or remove columns, handle null values, and implement specific business rules**. This is where we shape the raw data into a usable format for analysis.

- **Load**: The final step is to **store the transformed data in a target storage location**. This could be a data warehouse like Snowflake, BigQuery, or Redshift, or a simpler object storage solution such as Amazon S3, Google Cloud Storage, or Azure Blob Storage.

In essence, an ETL pipeline extracts data, transforms it according to business needs, and then loads it into a suitable destination for analysis.



3. Proposed Architecture Diagram Overview

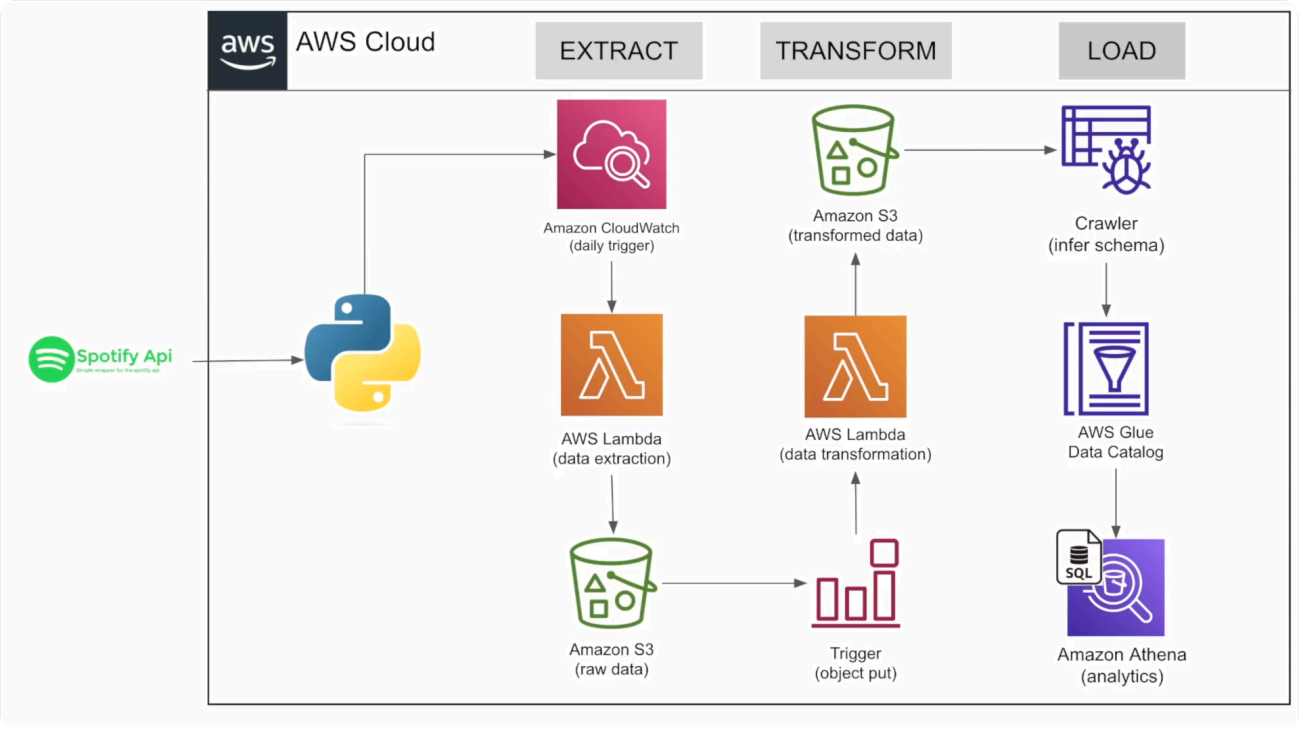
We have proposed a **completely serverless architecture** to our client, meaning we won't need to manage any servers ourselves. While the specific diagram isn't provided here, we will walk through each component and its role in detail.

The core flow involves:

1. **Extracting data from the Spotify API** on a weekly basis.
2. **Processing and transforming** that data.
3. **Storing and preparing it for analysis**.

We've chosen **AWS (Amazon Web Services)** for this project due to its comprehensive services and availability of a 12-month free trial for the services we'll use, ensuring no unexpected costs. However, it's important to note that similar services exist on other cloud platforms like Google Cloud Platform (GCP) and Microsoft Azure, and once you understand one, the concepts are largely transferable.

4. Detailed Architecture Components



Here are the key services and technologies we will integrate to build our pipeline:

- **Spotify API**:
  - This is our **primary data source**. Spotify provides an API that allows us to extract information about artists, albums, and individual tracks (songs).
  - We will **register our application with the Spotify API** to obtain a client ID and secret key, which are necessary for authentication.
  - We will use the `spotipy` **Python package** to simplify the interaction with the Spotify API and extract the required data.
- **AWS Services**:
  - **Amazon S3 (Simple Storage Service)**:
    - This is AWS's **highly scalable object storage service**. We will use S3 to **store both our raw extracted data and the transformed data**.
    - S3 is versatile, capable of storing any amount and type of data, including media files, backups, and even static websites.
    - It's comparable to Google Storage on GCP and Azure Blob Storage on Microsoft Azure.
  - **AWS Lambda**:
    - A **serverless computing service**. This means we can deploy our Python code without managing any underlying servers. AWS automatically handles the provisioning, scaling, and maintenance of the compute resources.
    - We will use Lambda functions for both the **data extraction and transformation jobs**.
  - **Amazon CloudWatch**:
    - A **monitoring service for AWS resources**.
    - We will use CloudWatch to **schedule and trigger our Lambda functions**. For instance, we will configure a CloudWatch trigger to run our data extraction Lambda function on a weekly basis, aligning with the client's requirement for weekly data collection. CloudWatch can also collect metrics, logs, and set alarms.
  - **AWS Glue Crawler**:

    - A **fully managed service that automatically crawls data sources** (like our S3 buckets).
    - It inspects files to **identify data formats, infers schemas (column names and data types), and then creates an AWS Glue Data Catalog**.
- **AWS Glue Data Catalog**:
    - This serves as a **central metadata repository for our data**. It stores information about our datasets, such as the number of columns, their names, data types, and where the files are stored on S3. This catalog is crucial for querying our data effectively.
- **Amazon Athena**:
    - A **serverless query service that allows us to run standard SQL queries directly on data stored in Amazon S3**.
    - Instead of setting up and managing a traditional database server, Athena enables direct querying of our CSV files (or other formats) on S3, leveraging the schema information from the AWS Glue Data Catalog. You only pay for the queries you run.

5. The ETL Pipeline Flow in Detail

Let's put these components together to understand the automated pipeline:

1. **Scheduled Extraction (E)**:
    - **Amazon CloudWatch** is configured with a **weekly trigger**.
    - This trigger invokes an **AWS Lambda function** dedicated to extraction.
    - This Lambda function uses the `spotipy` **Python package to connect to the Spotify API** and extract the latest top global songs data.
    - The extracted **raw data is then stored directly into an Amazon S3 bucket**.
2. **Automated Transformation (T)**:
    - Once new raw data lands in the S3 bucket, an **S3 event trigger automatically invokes another AWS Lambda function** designed for transformation.
    - This transformation Lambda function reads the raw data from S3, applies necessary **cleaning, restructuring, and business logic** (e.g., handling nulls, adding derived columns).
    - The **transformed data is then saved back into a different location (or folder) within Amazon S3**.
3. **Data Cataloging and Analytics (L)**:
    - After the transformed data is stored on S3, an **AWS Glue Crawler** is run.
    - The crawler scans the transformed data files in S3, **infers their schema (columns, data types)**, and populates this metadata into the **AWS Glue Data Catalog**.
    - With the data cataloged, analysts can use **Amazon Athena to directly run SQL queries** on the transformed data files in S3. This allows for efficient analysis of trending songs, albums, and artists without needing to load data into a separate database.

This entire process is designed to be **fully automated**. While this project focuses on a serverless AWS architecture, it's important to remember that such architectures are flexible. For example, you could replace AWS Lambda for orchestration with tools like Apache Airflow or commercial ETL tools like Alteryx or Informatica, depending on your specific requirements.
This project will provide you with a comprehensive understanding of building robust and scalable data pipelines, a critical skill for any data engineer.